

CS311: Lab 5 Pipelined CPU

컴퓨터공학과 20180806 류누리

컴퓨터공학과 20180285 이신범

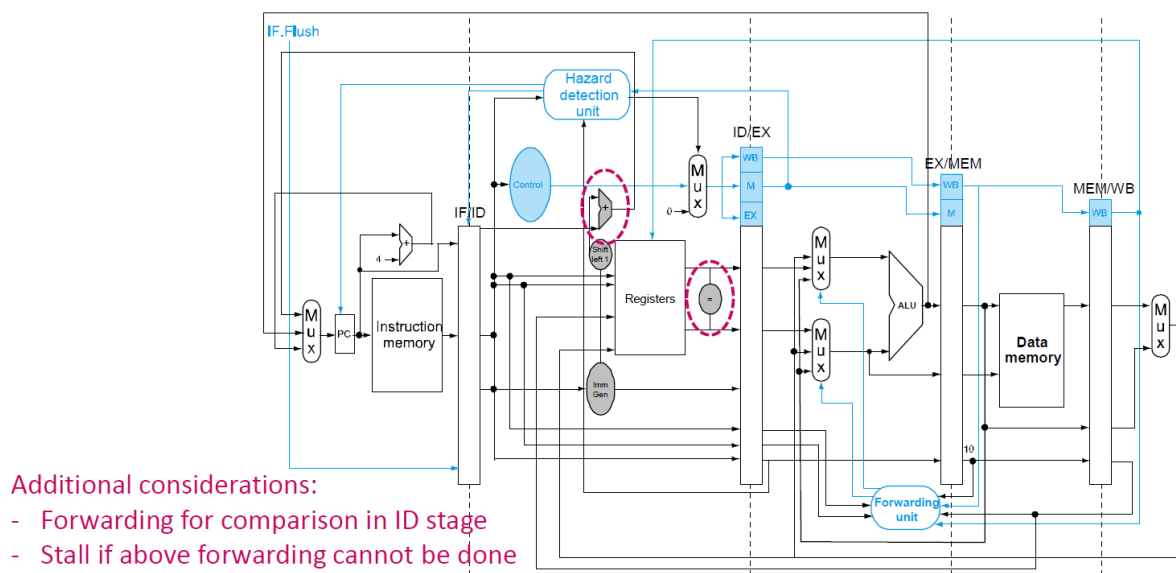
Introduction

이번 랩에서는 pipelined CPU를 Verilog를 활용하여 구현하는 것이 목표이다. pipelined CPU의 구성요소들과 그 작동방식을 이해하는 것이 이번 랩의 주 목표이다. HW를 약간 추가함으로써 지난번에 구현한 multi cycle CPU보다 throughput을 증가시키는 것이 목표이다. 또, pipeline CPU의 data & control hazard를 이해하고 이를 해결 할 방법을 구현해야한다. 즉, Data forwarding과 stall을 구현해야 한다. 또, branch prediction에 대한 개념을 이해하고 predictor를 구현해야 한다. Pipeline CPU의 개념들을 잘 이해하고 CPU의 구성요소를 module화 하여 코드를 작성하는 것이 최종목표이다.

Design

alu, alu_control, branch_control, branch_predictor, control, cpu, data_hazard_unit, pc_adder 총 8개의 모듈을 이용해서 always-not-taken이지만, BTB에 기록된 적이 있는 address면 taken을 하는, 파이프라인 cpu를 구현하였다.

Branch Resolved in ID Stage



Lecture 9의 branch resolved CPU를 참고하였다.

[alu] alu에서는 opcode와 funcCode를 이용해 ADD ~ SHR, ADI, ORI, LHI, LWD, SWD, JAL까지를 구현했다.

[alu_control] 이 때, alu 계산을 위해 필요한 A, B, Sign_Extend_Imm은 여러가지 신호를 통해 alu_control에서 구현했다. (forwarding을 제외한 신호는 ID_EX나 IF_ID에서 전달받는다)

[branch control] ALU를 사용하기 전, branch condition을 check 해주어야 하므로, 원래 alu에 구현해두었던 "bcond ="에 대한 내용을 branch control로 이동시켰다. (A, B, ID_EX_signal을 전달받는다)

[branch predictor] IF_predict_pc는 pc+1로 always-not-taken-predict 하되, target_address가 BTB에 저장되어 있다면, taken-predict를 하도록 구성하였다. 따라서, BTB의 개념을 조금 접목시킨 predictor라고 할 수 있다.

[control] ppt에 있는 그림을 참고하여 control logic을 만들었다. IF_ID_instruction으로부터 해당 instruction에 적용될 control 값들을 뽑아내고, 이것을 output wire [20:0] signal을 통해 cpu로 넘겨주었다. 이후, cpu에서는 clk에 따라 signal을 ID_EX_Signal로 옮겨준다.

[data_hazard_unit]

Forward A = rs register와 rd register가 걸리는 경우, EX/MEM hazard면 10, MEM_WB hazard면 01

Forward B = rt register와 rd register가 걸리는 경우, EX/MEM hazard면 10, MEM_WB hazard면 01

이 때, MEM_WB hazard인 경우, 이전에 EX/MEM hazard에 걸리지 않았어야 하므로, hazard를 검출하는 순서를 if / else if를 이용해 MEM/WB -> EX/MEM으로 해주었다.

[pc_adder]

그림에 나와있는 것처럼, branch prediction의 결과에 따른 target_address의 계산은 alu와 다른 module에서 수행되어야 하므로, 이를 위한 module을 구현해주었다.

Implementation

[cpu]

cpu에서는 다음과 같이 posedge clk일 때, ID/EX reg => EX/MEM reg => MEM/WB reg를 update 해주었다.

```

else begin
    flush <= 0;
    IF_ID_num_inst <= pc_num_inst;

    ID_EX_pc <= IF_ID_pc;
    ID_EX_predict_pc <= IF_ID_predict_pc;

    ID_EX_jump_target_address <= {IF_ID_pc[15:12], IF_ID_instruction[11:0]};

    ID_EX_rd <= IF_ID_instruction[7:6];
    ID_EX_rt <= rt;
    ID_EX_rs <= rs;
    ID_EX_operand_rs <= operand_rs;
    ID_EX_operand_rt <= operand_rt;
    ID_EX_Sign_Extended_Imm <= Sign_Extended_Imm;
    ID_EX_is_halted <= current_is_halted;

    ID_EX_num_inst <= IF_ID_num_inst;
    ID_EX_signal <= signal;
end

```

[IF/ID reg => ID/EX reg update]

```

EX_MEM_pc <= ID_EX_pc;
EX_MEM_alu_result <= C;
EX_MEM_operand_rt <= ID_EX_operand_rt;

EX_MEM_signal <= ID_EX_signal[10:4];

mem_wb_pc <= EX_MEM_pc;

mem_wb_alu_result <= EX_MEM_alu_result;
mem_wb_read_data <= data2;

mem_wb_signal <= {EX_MEM_signal[6:5], EX_MEM_signal[2:0]};

```

[ID/EX reg => EX/MEM reg => MEM/WB reg update]

이 때, control module을 통해 전달받은 signal은, 그 특성이 다른 variable과 비교했을 때 명확하므로, 하나의 긴 register를 통해 한꺼번에 옮겨주었다.

Signal, ID_EX_Signal, EX_MEM_Signal, MEM_WB_Signal의 구조는 다음과 같다.

	signal	id_ex	ex_mem	mem_wb
0	branch	branch	pc_to_reg	pc_to_reg
1	rtype_jump	rtype_jump	mem_to_reg	mem_to_reg
2	jtype_jump	jtype_jump	reg_write	reg_write
3	is_wwd	is_wwd	mem_read	reg_dest
4	pc_to_reg	pc_to_reg	mem_write	reg_dest
5	mem_to_reg	mem_to_reg	reg_dest	
6	reg_write	reg_write	reg_dest	
7	mem_read	mem_read		
8	mem_write	mem_write		
9	reg_dest	reg_dest		
10	reg_dest	reg_dest		
11	func_code	func_code		
12	func_code	func_code		
13	func_code	func_code		
14	func_code	func_code		
15	func_code	func_code		
16	func_code	func_code		
17	opcode	opcode		
18	opcode	opcode		
19	opcode	opcode		
20	opcode	opcode		

signal register elements

[internal forwarding]

internal forwarding의 경우, register update를 posedge, negedge로 나눌 것 없이, wire assign을 통해 "mem_wb_signal[4:3] == rs 혹은 rt"라면 wb값을 바로 전달함으로써 해결할 수 있었다. (이 때, JAL, JRL 혹은 load operation이 아니라면, wb == mem_wb_alu_result가 된다)

```
/* internal forwarding */
assign operand_rs = (mem_wb_signal[2] && (rs) == mem_wb_signal[4:3]) ? wb : regs[rs];
assign operand_rt = (mem_wb_signal[2] && (rt) == mem_wb_signal[4:3]) ? wb : regs[rt];
```

[Load-Use stall]

```
/* for load-use hazard */
assign IF_ID_Func_Code = IF_ID_instruction[5:0];
assign IF_ID_Op_Code = IF_ID_instruction[15:12];
assign use_rs = (IF_ID_Op_Code != 6) && (IF_ID_Op_Code != 9) && (IF_ID_Op_Code != 10);
assign use_rt = (IF_ID_Op_Code == 7) || (IF_ID_Op_Code == 8) || (IF_ID_Op_Code == 0) || (IF_ID_Op_Code == 1) || (IF_ID_Op_Code == 15) && (IF_ID_Func_Code == 0);

is_stall = ((rs == ID_EX_signal[10:9] && (use_rs)) || (rt == ID_EX_signal[10:9] && (use_rt))) && ID_EX_signal[7];
if (ID_EX_signal[2] && (ID_EX_predict_pc != ID_EX_jump_target_address)) is_stall = 1;
else if (ID_EX_signal[1] && (ID_EX_predict_pc != A)) is_stall = 1;
else if (ID_EX_signal[0] && bcond && (ID_EX_predict_pc != target)) is_stall = 1;
else if (ID_EX_signal[0] && ~bcond && (ID_EX_predict_pc != ID_EX_pc)) is_stall = 1;
// $display("rs: %d, rt: %d, dest: %d, Load?: %d, is_stall?: %d, IF_ID_Op_Code: %d, IF_ID_Func_Code: %d, output: %h", rs, rt, ID_EX_signal[10:9],
```

Load_use stall의 경우, 다음과 같이 1) 앞선 명령이 "load"이며, 2) id_ex_rd == rs 혹은 rt 인 경우 stall하는 condition을 추가해줌으로써 해결할 수 있었다.

```

end
else if (((rs == ID_EX_signal[10:9] && (use_rs)) || (rt == ID_EX_signal[10:9] && (use_rt))) && ID_EX_signal[7]) begin
    pc <= ID_EX_pc;
end

```

또한, 이 경우 ID_EX_pc를 pc에 넣어줌으로써 같은 명령을 한 번 delay 시키는 효과를 주었다.

[flush & stall]

```

if(is_stall) begin
    flush <= 1;

    /* ID_EX_pc <= ID_EX_pc; */
    ID_EX_predict_pc <= 0;

    ID_EX_rd <= 0;
    ID_EX_rt <= 0;
    ID_EX_rs <= 0;
    ID_EX_operand_rs <= 0;
    ID_EX_operand_rt <= 0;
    ID_EX_Sign_Extended_Imm <= 0;
    ID_EX_is_halted <= 0;

    pc_num_inst <= IF_ID_num_inst;
    IF_ID_num_inst <= ID_EX_num_inst;
    ID_EX_num_inst <= ID_EX_num_inst;
    ID_EX_signal <= 21'b0;
end

```

<=stall / flush ==>

```

else begin
    Func_Code = 0;
    Reg_Dest = 0;
    Mem_Write = 0;
    Mem_Read = 0;
    Reg_Write = 0;
    Mem_to_Reg = 0;
    Pc_to_Reg = 0;
    Is_Halted = 0;
    Is_WWD = 0;
    J_Type_Jump = 0;
    R_Type_Jump = 0;
    branch = 0;
end

```

Flush는, 만약 flush == 1인 경우, "condition.v"에서 signal을 모두 '0'으로 처리함으로써 새로운 명령이 저장되지 못하게 하였다. Stall은, 다음과 같이 ID_EX_signal을 모두 '0'으로 처리하고, num_inst를 한 번 delay 시켰다.

```

is_stall = ((rs == ID_EX_signal[10:9] && (use_rs)) || (rt == ID_EX_signal[10:9] && (use_rt))) && ID_EX_signal[7];
if (ID_EX_signal[2] && (ID_EX_predict_pc != ID_EX_jump_target_address)) is_stall = 1;
else if (ID_EX_signal[1] && (ID_EX_predict_pc != A)) is_stall = 1;
else if (ID_EX_signal[0] && bcond && (ID_EX_predict_pc != target)) is_stall = 1;
else if (ID_EX_signal[0] && ~bcond && (ID_EX_predict_pc != ID_EX_pc)) is_stall = 1;
//display("rs: %d, rt: %d, dest: %d, Load?: %d, is_stall?: %d, IF_ID_Op_Code: %d, IF_ID_Func_Code: %d, output: %h

```

Stall 및 flush의 condition은 다음과 같다.

Discussion

Lab 4에서의 multi cycle CPU에서는 총 8526 clock cycle이 필요했으며, 이번 랩에서 구현한 pipelined CPU의 경우 Always not taken의 경우 총 1437 clock cycle이 필요했다. 즉, 5.933배 더 빠른 성능을 보여주었다. Branch predictor를 사용한 경우 총 1229 clock cycle이 필요했다. 즉 Always not taken pipelined CPU에 비해 16.9%의 성능향상을 보여주었다.

```
# Clock # 8526
# The testbench is finished. Summarizing...
# All Pass!
# ** Note: $finish      : C:/Users/ryunu/Desktop/lab/lab4/cpu_TB.v(152)
#      Time: 852750 ns  Iteration: 2  Instance: /cpu_TB
```

<Lab4 Multi cycle CPU의 clock cycle>

```
# Clock # 1437
# The testbench is finished. Summarizing...
# All Pass!
# ** Note: $finish      : C:/Users/ryunu/Desktop/lab/lab5/different version/cpu_TB.v(153)
#      Time: 143850 ns  Iteration: 2  Instance: /cpu_TB
```

<Lab5 Pipeline CPU, always not taken의 clock cycle>

```
# Clock # 1229
# The testbench is finished. Summarizing...
# All Pass!
# ** Note: $finish      : C:/Users/ryunu/Desktop/lab/lab5/different version/cpu_TB.v(153)
#      Time: 123050 ns  Iteration: 2  Instance: /cpu_TB
```

<Lab5 Pipeline CPU, always taken의 clock cycle>

하지만 이론상 pipeline CPU에서도 불가피한 stall이나 flush가 존재하며, multi cycle CPU와 달리 모든 instruction이 5 cycle을 소모하기 때문에, 이 정도의 performance 향상이 나올지는 몰랐다. 아마 lab4의 multi cycle CPU의 구현에 우리가 놓친 것이 있는 것 같다.

이번 랩 구현에 있어서 우리는 특히 data/control hazard를 resolve하는 부분을 구현하는 것이 힘들었다. 하지만 결론적으로는 stall condition을 파악하여 구현해냈다.

Conclusion

결론적으로, 우리는 pipelined TSC CPU를 구현할 수 있었으며, test bench가 요구하는 사항을 모두 충족시킬 수 있었다. 이번 랩을 통해서 pipelined CPU의 작동원리에 대해서 배울 수 있었다.